

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS ARARANGUÁ
DEPARTAMENTO DE TECNOLOGIA DA INFORMAÇÃO E COMUNICAÇÃO
CURSO TECNOLOGIAS DA INFORMAÇÃO E COMUNICAÇÃO (TIC)

Josimar Bitencourt Pereira
Leonardo Alves dos Santos

Relatório de Manutenção e Evolução de Software
Projeto app_gestao_frotas

Engenharia de Software II

Araranguá
2026

1 INTRODUÇÃO

A manutenção e a evolução de software correspondem a uma parcela significativa do ciclo de vida de um sistema, pois exigem compreender, avaliar e modificar um código desenvolvido por terceiros sem comprometer as funcionalidades já existentes. Neste contexto, o presente relatório descreve o trabalho de manutenção e refatoração realizado sobre o projeto de código aberto `app_gestao_frotas`.

O repositório base pertence a Fabricio Leonard (fabricioLeonard / FabricioLeonardFIEG) e foi clonado para a realização desta atividade. O trabalho foi organizado em duas branches: `main`, que preserva a linha de base original, e `manutencao`, na qual foram aplicadas a evolução e a refatoração. Os desenvolvedores responsáveis foram Josimar Bitencourt Pereira (Josimar-Pereira) e Leonardo Vinci (VinciLeonardo).

O objetivo é documentar o estado inicial do sistema, apontar as falhas arquiteturais identificadas, apresentar um comparativo entre o código antes e depois das alterações e registrar o histórico de commits que garante a rastreabilidade do processo.

2 CARACTERIZAÇÃO E COMPREENSÃO INICIAL DO SISTEMA

2.1 Objetivo do software

O `app_gestao_frotas` é uma aplicação em Java para console projetada para gerenciar uma frota multimodal de veículos (carros, motos, caminhões, ônibus, ambulâncias, tratores e triciclos elétricos). O sistema fornece funcionalidades para cadastro, persistência em banco de dados relacional leve (SQLite local, arquivo `frota.db`) e listagem polimórfica das fichas técnicas no terminal.

2.2 Tecnologias utilizadas

As tecnologias empregadas no projeto são listadas a seguir:

a) linguagem base: Java 21 (JDK 21), com uso de Pattern Matching (`instanceof`), Switch Expressions e Text Blocks;
b) gerenciamento de build e dependências: Apache Maven 3.x;
c) persistência de dados: SQLite JDBC (sqlite-jdbc v3.45.1.0), por meio de `PreparedStatement`;

d) testes automatizados: JUnit 5 (junit-jupiter v5.10.2) e AssertJ (assertj-core v3.25.3);
e) controle de versão: Git e GitHub.

3 BASELINE DO PROJETO

Na etapa inicial de clonagem e inspeção do repositório original, realizada na branch `main`, estabeleceu-se a linha de base para a auditoria do código, conforme descrito a seguir:

a) branch base: `main`;
b) compilação Maven: concluída com sucesso (`mvn clean compile`);
c) cobertura de testes automatizados inicial: 0%, devido à ausência total de suíte de testes em `src/test/java`;
d) status de funcionalidade: execução interativa via menu limitada ao cadastro de carros e motos e à listagem da frota.

4 ANÁLISE DE SMELLS, ARQUITETURA E OPORTUNIDADES DE MELHORIA

Durante a fase de inspeção do código e análise de manutenibilidade, foram identificadas as falhas arquiteturais descritas nas subseções seguintes.

4.1 Redundância e duplicação no ponto de entrada

Diagnóstico: existência de duas classes de execução concorrentes, `Main.java` (classe estática curta) e `MainOriginal.java` (classe contendo o menu interativo). Trata-se do code smell *Duplicate Code*.

Impacto: comprometia a clareza do ponto de entrada da aplicação na JVM.

4.2 Violação do Princípio da Responsabilidade Única em ConexaoBanco

Diagnóstico: a classe `ConexaoBanco` concentrava responsabilidades de criação de schema (DDL), operações SQL (DML via JDBC) e validação direta de regras de negócio do domínio, como faixas de ano, limites de portas e de cilindradas.

Impacto: dificultava a realização de testes unitários isolados e misturava infraestrutura com regras de negócio.

4.3 Alto acoplamento entre interface de usuário e persistência

Diagnóstico: a classe de interface de usuário (`MainOriginal`) invocava diretamente métodos estáticos do banco de dados (`ConexaoBanco.salvarVeiculo`).

Impacto: dificultava a substituição da tecnologia de banco de dados ou a transição para uma API.

4.4 Incompletude nas operações de persistência (CRUD)

Diagnóstico: o sistema possuía apenas funcionalidades de inserção (`salvarVeiculo`) e de consulta (`listarVeiculos`).

Impacto: impossibilidade de excluir veículos já cadastrados.

5 COMPARATIVO ENTRE O ESTADO INICIAL E O ESTADO FINAL

5.1 Visão geral das alterações

A Tabela 1 sintetiza as alterações realizadas, relacionando cada componente ao seu estado inicial (branch `main`), ao seu estado final (branch `manutencao`) e à justificativa arquitetural adotada.

Tabela 1 – Comparativo “antes e depois” do código

Componente / camada	**Estado inicial (antes – `main`)**	**Estado final (depois – `manutencao`)**	**Justificativa arquitetural e padrões aplicados**
Ponto de entrada	Dividido entre `Main.java` e `MainOriginal.java`.	Unificado exclusivamente na classe `Main.java` (`MainOriginal.java` removido).	DRY (*Don't Repeat Yourself*): elimina a redundância e consolida o ponto de entrada da aplicação.
Regras de validação	Inseridas no método `salvarVeiculo`, em `ConexaoBanco.java`.	Extraídas para a classe especializada `ValidadorDeVeiculo`.	SRP (Princípio da Responsabilidade Única): isola o domínio do código de infraestrutura JDBC.
Arquitetura em camadas	Comunicação direta de `Main` com a classe `ConexaoBanco`.	Criação da camada de serviço intermediária `FrotaService.java`.	Baixo acoplamento: desacopla a interface de console dos detalhes de banco de dados.
Operação de exclusão	Ausente no sistema.	Método `deletarVeiculo(int id)` adicionado ao DAO, ao Service e à opção 9 do menu.	Manutenção perfectiva: evolução do ciclo de vida dos dados (exclusão por ID).

Componente / camada	Estado inicial (antes – `main`)	Estado final (depois – `manutencao`)	Justificativa arquitetural e padrões aplicados
Testes automatizados	Ausentes.	Classes `VeiculoTest` e `ConexaoBancoTest`, com JUnit 5.	Garantia de qualidade e regressão: assegura a consistência dos modelos e das exceções de negócio.

Fonte: Elaborado pelos autores (2026).

5.2 Validação e persistência de dados

O Código 1 apresenta o método `salvarVeiculo` na linha de base: as regras de negócio (validação de ano e de marca) estavam misturadas ao código SQL dentro de `ConexaoBanco`.

Código 1 – Antes: `ConexaoBanco.java` acoplada às regras de negócio

```
// Código herdado da baseline: regras de negócio misturadas com SQL
public static void salvarVeiculo(Veiculo veiculo)
        throws SQLException, DadosVeiculoInvalidosException {

    if (veiculo.getAno() < 1900 || veiculo.getAno() > 2027) {
        throw new DadosVeiculoInvalidosException(
                "Ano do veículo é inválido: " + veiculo.getAno());
    }
    if (veiculo.getMarca() == null || veiculo.getMarca().isBlank()) {
        throw new DadosVeiculoInvalidosException("A marca deve ser informada!");
    }

    // ... DML SQL
}
```

Fonte: Repositório base, adaptado pelos autores (2026).

Na versão evoluída, as validações foram isoladas em `ValidadorDeVeiculo`, que passa a pertencer ao domínio (pacote `model`), conforme o Código 2.

Código 2 – Depois: `ValidadorDeVeiculo.java` com a validação isolada

```java
package br.com.gestaofrota.model;

import br.com.gestaofrota.excecoes.DadosVeiculoInvalidosException;

public class ValidadorDeVeiculo {

    public static void validar(Veiculo veiculo)
            throws DadosVeiculoInvalidosException {

        if (veiculo.getAno() < 1900 || veiculo.getAno() > 2027) {
            throw new DadosVeiculoInvalidosException(
                    "Ano do veículo é inválido: " + veiculo.getAno());
        }
        if (veiculo.getMarca() == null || veiculo.getMarca().isBlank()) {
            throw new DadosVeiculoInvalidosException(
                    "A marca deve ser informada!");
        }
        if (veiculo.getModelo() == null || veiculo.getModelo().isBlank()) {
            throw new DadosVeiculoInvalidosException(
                    "O modelo deve ser informado!");
        }
        if (veiculo instanceof Carro c) {
            if (c.getQuantidadePortas() < 2 || c.getQuantidadePortas() > 6) {
                throw new DadosVeiculoInvalidosException(
                        "Quantidade de portas deve ser de 2 a 6!");
            }
        }
        if (veiculo instanceof Moto m) {
            if (m.getCilindradas() < 50 || m.getCilindradas() > 2500) {
                throw new DadosVeiculoInvalidosException(
                        "Quantidade de cilindradas deve ser entre 50 e 2500!");
            }
        }
    }
}
```

Fonte: Elaborado pelos autores (2026).

Com isso, `ConexaoBanco` passou a ter foco exclusivo na infraestrutura JDBC, além de incorporar a exclusão por ID, como mostram os Códigos 3, 4 e 5.

Código 3 – Depois: `ConexaoBanco.java` – conexão e criação da tabela

```
package br.com.gestaofrota.database;

import br.com.gestaofrota.model.*;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ConexaoBanco {

    private static final String URL_CONEXAO = "jdbc:sqlite:frota.db";

    public static Connection conectar() throws SQLException {
        return DriverManager.getConnection(URL_CONEXAO);
    }

    public static void inicializarTabela() {
        String sql = """
                CREATE TABLE IF NOT EXISTS veiculos (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    tipo TEXT NOT NULL,
                    marca TEXT NOT NULL,
                    modelo TEXT NOT NULL,
                    ano INTEGER NOT NULL,
                    detalhe_especifico INTEGER NOT NULL
                );
                """;
        try (Connection conn = conectar();
             Statement stmt = conn.createStatement()) {
            stmt.execute(sql);
        } catch (SQLException e) {
            System.err.println("Erro ao inicializar o banco de dados: "
                    + e.getMessage());
        }
    }

    // ... demais métodos nos Códigos 4 e 5
}
```

Fonte: Elaborado pelos autores (2026).

Código 4 – Depois: `ConexaoBanco.java` – inserção de veículos

```
    // ConexaoBanco.java (continuação)
    public static void salvarVeiculo(Veiculo veiculo) throws SQLException {
        String sql = "INSERT INTO veiculos(tipo, marca, modelo, ano, "
                + "detalhe_especifico) VALUES(?, ?, ?, ?, ?)";
        try (Connection conn = conectar();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, veiculo.getClass().getSimpleName().toUpperCase());
            stmt.setString(2, veiculo.getMarca());
            stmt.setString(3, veiculo.getModelo());
            stmt.setInt(4, veiculo.getAno());

            if (veiculo instanceof Carro c)
                stmt.setInt(5, c.getQuantidadePortas());
            else if (veiculo instanceof Moto m)
                stmt.setInt(5, m.getCilindradas());
            else if (veiculo instanceof Caminhao c)
                stmt.setDouble(5, c.getCapacidadeCargaToneladas());
            else if (veiculo instanceof Onibus o)
                stmt.setInt(5, o.getCapacidadePassageiros());
            else if (veiculo instanceof Ambulancia a)
                stmt.setInt(5, a.getNumeroUTI());
            else if (veiculo instanceof TricicloEletrico t)
                stmt.setInt(5, t.getAutonomiaBateriaKm());
            else if (veiculo instanceof Trator t)
                stmt.setInt(5, t.getHorasUso());

            stmt.executeUpdate();
        }
    }
```

Fonte: Elaborado pelos autores (2026).

Código 5 – Depois: `ConexaoBanco.java` – listagem e exclusão por ID

```
    // ConexaoBanco.java (continuação)
    public static List<Veiculo> listarVeiculos() throws SQLException {
        List<Veiculo> frota = new ArrayList<>();
        String sql = "SELECT * FROM veiculos";

        try (Connection conn = conectar();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {

            while (rs.next()) {
                int id = rs.getInt("id");
                String tipo = rs.getString("tipo");
                String marca = rs.getString("marca");
                String modelo = rs.getString("modelo");
                int ano = rs.getInt("ano");
                int detalhe = rs.getInt("detalhe_especifico");

                if ("CARRO".equalsIgnoreCase(tipo))
                    frota.add(new Carro(id, marca, modelo, ano, detalhe));
                else if ("MOTO".equalsIgnoreCase(tipo))
                    frota.add(new Moto(id, marca, modelo, ano, detalhe));
                else if ("CAMINHAO".equalsIgnoreCase(tipo))
                    frota.add(new Caminhao(id, marca, modelo, ano, detalhe));
                else if ("AMBULANCIA".equalsIgnoreCase(tipo))
                    frota.add(new Ambulancia(id, marca, modelo, ano, detalhe));
                else if ("ONIBUS".equalsIgnoreCase(tipo))
                    frota.add(new Onibus(id, marca, modelo, ano, detalhe));
                else if ("TRICICLOELETRICO".equalsIgnoreCase(tipo))
                    frota.add(new TricicloEletrico(id, marca, modelo, ano,
                            detalhe));
                else if ("TRATOR".equalsIgnoreCase(tipo))
                    frota.add(new Trator(id, marca, modelo, ano, detalhe));
            }
        }
        return frota;
    }

    // Funcionalidade de manutenção perfectiva
    public static boolean deletarVeiculo(int id) throws SQLException {
        String sql = "DELETE FROM veiculos WHERE id = ?";
        try (Connection conn = conectar();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, id);
            return stmt.executeUpdate() > 0;
        }
    }
}
```

Fonte: Elaborado pelos autores (2026).

5.3 Camada de serviço intermediária

A classe `FrotaService`, apresentada no Código 6, atua como componente de desacoplamento: a interface de console passa a conhecer apenas o serviço, que coordena a validação e a persistência.

Código 6 – Depois: `FrotaService.java`

```java
package br.com.gestaofrota.service;

import br.com.gestaofrota.database.ConexaoBanco;
import br.com.gestaofrota.excecoes.DadosVeiculoInvalidosException;
import br.com.gestaofrota.model.ValidadorDeVeiculo;
import br.com.gestaofrota.model.Veiculo;
import java.sql.SQLException;
import java.util.List;

public class FrotaService {

    public void cadastrarVeiculo(Veiculo veiculo)
            throws SQLException, DadosVeiculoInvalidosException {
        ValidadorDeVeiculo.validar(veiculo);
        ConexaoBanco.salvarVeiculo(veiculo);
    }

    public List<Veiculo> listarFrota() throws SQLException {
        return ConexaoBanco.listarVeiculos();
    }

    public boolean excluirVeiculo(int id) throws SQLException {
        return ConexaoBanco.deletarVeiculo(id);
    }
}
```

Fonte: Elaborado pelos autores (2026).

5.4 Suíte de testes automatizados

Foram criados testes com JUnit 5 para validar as exceções de domínio (Código 7) e o comportamento polimórfico da ficha técnica dos veículos (Código 8).

Código 7 – Depois: `ConexaoBancoTest.java` – validação de exceções de domínio

```java
package br.com.gestaofrota.database;

import br.com.gestaofrota.excecoes.DadosVeiculoInvalidosException;
import br.com.gestaofrota.model.Carro;
import br.com.gestaofrota.model.ValidadorDeVeiculo;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ConexaoBancoTest {

    @Test
    public void testValidacaoAnoInvalido() {
        Carro carroInvalido = new Carro("Fiat", "Palio", 1880, 4);

        Exception exception = assertThrows(
                DadosVeiculoInvalidosException.class, () -> {
                    ValidadorDeVeiculo.validar(carroInvalido);
                });

        assertTrue(exception.getMessage()
                .contains("Ano do veículo é inválido"));
    }
}
```

Fonte: Elaborado pelos autores (2026).

Código 8 – Depois: `VeiculoTest.java` – polimorfismo e ficha técnica

```java
package br.com.gestaofrota.model;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class VeiculoTest {

    @Test
    public void testExibirFichaTecnicaCarro() {
        Carro carro = new Carro(1, "Fiat", "Uno", 2015, 4);

        String ficha = carro.exibirFichaTecnica();

        assertNotNull(ficha);
        assertTrue(ficha.contains("CARRO"));
        assertTrue(ficha.contains("Fiat"));
        assertTrue(ficha.contains("Uno"));
        assertTrue(ficha.contains("2015"));
        assertTrue(ficha.contains("Portas: 4"));
    }
}
```

Fonte: Elaborado pelos autores (2026).

6 HISTÓRICO DE COMMITS

A Tabela 2 apresenta o histórico de commits do repositório, em ordem cronológica inversa, o que assegura a rastreabilidade das alterações. As datas não foram informadas para os commits da branch `manutencao`, exceto o de clonagem.

Tabela 2 – Histórico de commits do repositório

Etapa	Mensagem do commit	Autor	Data
Manutenção e refatoração (`manutencao`)	docs: atualizar README.md com nova arquitetura, SRP, FrotaService e instrucoes do JUnit 5	Josimar-Pereira	-
Manutenção e refatoração (`manutencao`)	refactor: isolar persistencia em ConexaoBanco e atualizar Main para consumir FrotaService	VinciLeonardo	-
Manutenção e refatoração (`manutencao`)	refactor: aplicar SRP separando validacao em ValidadorDeVeiculo e desacoplando com FrotaService	Josimar-Pereira	-
Manutenção e refatoração (`manutencao`)	refactor: consolidar menu na classe Main e remover arquivo redundante MainOriginal	Josimar-Pereira	-
Manutenção e refatoração (`manutencao`)	removendo pasta docs	VinciLeonardo	-
Manutenção e refatoração (`manutencao`)	chore: adiciona JUnit 5, AssertJ e Surefire ao pom.xml	VinciLeonardo	-
Clonagem	Clonando repositório open source para o trabalho de Manutenção e Evolução de um projeto Open Source	VinciLeonardo	02/10/2026
Projeto base	feat: adicionado o código com a correção da 02, com exeção do Exercício 2 [Tratamento e Validação com Exceções Customizadas]	fabricioLeonard	10/09/2026
Projeto base	feat: adicionado a lista de exercícios com gabarito	fabricioLeonard	03/09/2026
Projeto base	feat: adicionado a lista de exercícios com gabarito	fabricioLeonard	27/08/2026
Projeto base	feat: adicionado o caminhão e validação	fabricioLeonard	27/08/2026
Projeto base	feat: sistema de gestão de frota com persistência SQLite	FabricioLeonardFIEG	20/08/2026

Fonte: Histórico do repositório Git (2026).

7 CONCLUSÃO

A experiência prática de analisar, refatorar e evoluir um software de código aberto desenvolvido por terceiros permitiu vivenciar desafios reais da Engenharia de Software. A falta de testes automatizados e o acoplamento inicial exigiram cautela para não introduzir quebras nas funcionalidades existentes.

Ao aplicar o Princípio da Responsabilidade Única (SRP) por meio da classe `ValidadorDeVeiculo`, isolar a camada de acesso a dados (`ConexaoBanco`) e introduzir o serviço `FrotaService`, a aplicação tornou-se modular, testável e de fácil manutenção. A inclusão dos testes com JUnit 5 garantiu a estabilidade e a prevenção contra regressões.

REFERÊNCIAS

LEONARD, Fabricio. **app_gestao_frotas**: sistema de gestão de frota com persistência SQLite. [S. l.]: GitHub, 2026. Repositório de código. Disponível em: https://github.com/fabricioLeonard/app_gestao_frotas.git. Acesso em: 20 out. 2026.